Phase Erasure Can Hide Agreement-Violation Witnesses in Certificate-Driven Locking BFT Protocols

Abstract. Fault-injection frameworks for Byzantine fault-tolerant protocols expose different amounts of control over message delivery. We characterize when closing phase-specific admission to one type-uniform directed endpoint graph preserves finite agreement safety and when closure-added repair evidence disables a unique correct quorum pivot, deleting the violation witness. We give a legal relational preservation theorem, a structural separation criterion, a parametric $3f + 1$ family, a bounded Tendermint-style rule-level instantiation, and a replay-erasure theorem requiring state and output congruence.

Best student paper eligibility: not specified in the research brief.

Phase Erasure Can Hide Agreement-Violation Witnesses in Certificate-Driven Locking BFT Protocols

Anonymous author
Anonymous affiliation

Abstract

Fault-injection frameworks for Byzantine fault-tolerant protocols expose different amounts of control over message delivery. Some choose one partition or directed endpoint graph for an entire view, whereas others distinguish message phases, individual instances, or replay. We study closure from phase-specific admission to one type-uniform directed endpoint graph for finite certificate-driven locking systems. A legal relational theorem gives sufficient conditions for preserving an unsafe terminal state. A structural quorum-pivot criterion and a parametric $3f + 1$ family show the boundary: closure-added repair evidence can disable the unique correct contributor required by two conflicting quorums. We instantiate the mechanism in bounded Tendermint-style rules under a controlled missing-lock mutation. A secondary theorem proves exact replay erasure only under state and output congruence, duplicate stutter, and silent output. No production-client vulnerability, universal hierarchy, or liveness result is claimed.

2012 ACM Subject Classification Theory of computation → Distributed algorithms; Computer systems organization → Dependable and fault-tolerant systems and networks; Software and its engineering → Formal methods

Keywords and phrases Byzantine fault tolerance, consensus testing, scheduler semantics, communication closure, safety, formal methods

Digital Object Identifier 10.4230/LIPIcs.OPODIS.2026.0

1 Introduction

Byzantine fault-tolerant (BFT) protocols are routinely evaluated under adversarial message schedules. The testing harness decides which messages are delivered, withheld, delayed, reordered, or replayed, and therefore determines which parts of the adversarial execution language are explored. Twins realizes several Byzantine behaviors with replicas that share one identity and receive different communication views [2]. Randomized BFT testing extends the fault language with message mutation and richer delivery choices [13]. Netrix exposes programmable predicates and actions over messages and executions [8], while ByzzBench supplies controlled benchmark infrastructure for comparing testing algorithms [1].

These systems establish that richer controls are useful. They do not by themselves answer a property-preservation question. Suppose a phase-specific policy can deliver a `PREVOTE` on an edge while withholding a later `PRECOMMIT` on that same edge, whereas a coarser policy assigns one Boolean admission decision to the endpoint pair for the whole view. The coarse policy necessarily admits both phases or neither. Does this loss of control merely collapse redundant traces, or can it remove an agreement-violation observation?

Raw trace inclusion is insufficient. A per-instance scheduler produces more schedules than a per-view graph almost by definition. The meaningful object is the image of those executions under a safety observation. Two grammars can differ on internal traces yet expose exactly the same conflicting decisions. Conversely, adding messages during closure can change protocol state and repair an execution that was unsafe before closure. This paper isolates the mechanism responsible for the latter case.


© Anonymous author(s);
licensed under Creative Commons License CC-BY 4.0



The central mechanism is a **unique correct quorum pivot**. Two conflicting certificate opportunities each have $q-1$ base contributions and need the same correct validator p. An enabling envelope E makes p available to the first branch. A later repair envelope R, sent on the same endpoint in the same view but with a different phase label, disables p before the second branch. A phase-specific policy admits E and withholds R, so both certificates form. A type-uniform directed policy has only two choices: with the edge off, the first certificate remains short; with the edge on, repair arrives before the second opportunity and the second certificate remains short.

The result is not a claim that graph-based schedules are universally incomplete. It is an existence and characterization result for an explicit finite class. Communication-closure work already proves that certain asynchronous consensus executions can be normalized to lossy synchronous executions under stated implementation conditions [7]. Heard-Of and message-adversary theories already treat round communication graphs as first-class semantic objects [4, 12]. Our question is narrower: when closure from phase-labelled admission to one endpoint bit preserves finite agreement violations, and when closure-added repair evidence destroys the witness.

1.1 Contributions

The paper makes four ordered contributions.

1. **Phase-erasure boundary.** We identify the repair/pivot mechanism by which closure from phase-specific admission to a type-uniform directed endpoint graph deletes an agreement-violation witness.
2. **Legal relational preservation theorem.** We give sufficient conditions under which closure-added deliveries preserve an unsafe terminal state, explicitly requiring that the transformed trace is legal in the target scheduler language.
3. **Structural criterion, parametric family, and natural protocol bridge.** We state a protocol-independent quorum-pivot separation criterion, derive an explicit $n = 3f + 1$ family, characterize its seven Boolean premises, and instantiate the mechanism in bounded Tendermint-style locking rules pinned to the CometBFT v0.39.3 specification [5].
4. **Replay-erasure boundary.** As a secondary closure dimension, we prove that exact duplicate erasure preserves both relevant state and relevant output traces only under state and output congruence, projected duplicate stutter, and silent duplicate handling.

The SMT queries, finite checkers, Go/Python models, and prover sources are evidence for these contributions rather than separate intellectual contributions.

1.2 Scope

The scope is finite agreement safety in authenticated, certificate-driven locking schemas. We do not claim a vulnerability in CometBFT, HotStuff, Tendermint, or another deployed implementation; universal incompleteness of partition or directed grammars; universal minimality of phase-specific scheduling; a liveness theorem; or completeness under unbounded partial synchrony. The CometBFT connection is a bounded refinement to published transition rules under one explicit mutation, not a source-to-source proof for the production Go client.

2 Model and exact scheduler semantics

2.1 Protocol executions

A protocol is a labelled transition system

$$P = (S, s_0, M, A, \delta, \mathsf{emit}),$$

where S is the global protocol state, s_0 the initial state, M the set of message envelopes, A the set of local and delivery actions, $\delta : S \times A \to S$ the deterministic transition function inside the bounded model, and emit the finite sequence of envelopes produced by a transition. An envelope is

$$m = (id, v, \lambda, s, r, payload),$$

with stable instance identifier id, view v, phase/type label $\lambda \in \Lambda$, sender s, and receiver r. Cryptographic validity and protocol-side acceptance are part of P; the scheduler cannot synthesize a message that P did not emit.

A finite execution is

$$\tau = s_0\, a_1\, s_1 \cdots a_k\, s_k, \quad s_i = \delta(s_{i-1}, a_i).$$

We write $\mathrm{last}(\tau) = s_k$. A bound descriptor B fixes the participants, values, views, phase order, message-production opportunities, scheduler-controlled edges, and any permitted replay. The legal language of protocol P under grammar G and bound B is $L_G(P, B)$.

2.2 Safety observations and typing

Let

$$O : S \to \Omega$$

be the observation function. For agreement, $O(s)$ records the decisions of correct identities in state s. Let

$$\mathsf{Unsafe} : S \to \{\mathsf{true}, \mathsf{false}\}$$

be a predicate on protocol states, not on observations. An execution observation is

$$\mathrm{Obs}(\tau) = O(\mathrm{last}(\tau)).$$

The unsafe observation language is therefore

$$\mathsf{UnsafeObs}_G(P, B) = \{O(\mathrm{last}(\tau)) \mid \tau \in L_G(P, B) \wedge \mathsf{Unsafe}(\mathrm{last}(\tau))\}.$$

This separation permits different unsafe states to map to the same observation without applying a state predicate to an element of Ω.

2.3 Admission grammars

Let V be the bounded set of views and N the identities. A type-uniform directed grammar is an admission function

$$A_D : V \times N \times N \to \{0, 1\}.$$

All controlled phases on edge (v, s, r) share one bit. A phase-specific grammar is

$$A_T : V \times \Lambda \times N \times N \to \{0, 1\}.$$

The type-erasure closure is

$$C_{\text{type}}(A_T)(v, s, r) = \bigvee_{\lambda \in \Lambda} A_T(v, \lambda, s, r).$$

Thus an endpoint edge is opened whenever the fine policy admits at least one controlled phase on it.

For an emitted envelope sequence $E = [m_1, \ldots, m_j]$, delivery under policy A is a **stable filter**:

$$\text{deliver}_A(E) = [m_i \mid A(m_i) = 1],$$

in the order in which the protocol emitted the envelopes. This definition has four consequences used by the theorems:

1. **no synthesis:** every delivered envelope occurs in E;
2. **at most once:** an instance identifier occurs no more often than in E unless replay is explicitly enabled by another grammar;
3. **order preservation:** if two admitted instances occur in that order in E, delivery preserves it;
4. **fixed delivery point in the bounded domain:** newly admitted envelopes are processed at their predetermined phase opportunity. The domain used for the separation contains no late delivery of repair after the second certificate opportunity.

Only edges declared controlled by B are filtered. All other emissions and deliveries are fixed. A per-instance grammar G_M and a replay grammar G_R are used only in the secondary replay result; the central separation compares G_T and G_D.

2.4 Closure as a legal trace transformation

For a fine policy A_T, closure constructs a candidate target trace $C(\tau)$ by retaining the original protocol actions and inserting deliveries admitted by $C_{\text{type}}(A_T)$ that were withheld by A_T. Closure never invents an envelope or changes the relative order of original actions. This sequence is a semantic target only if

$$C(\tau) \in L_{G_D}(P, B).$$

The legality obligation is independent of the relational preservation proof. A target grammar may forbid an insertion, a receiver may reject it, or its processing may invalidate the remaining action sequence. The next section therefore separates merged-trace reasoning from target-language legality.

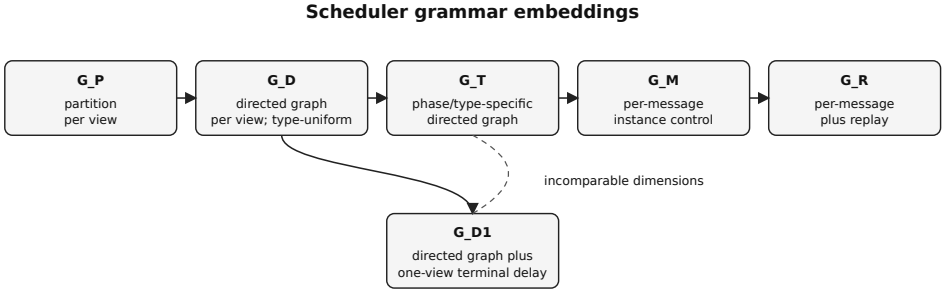


Figure 1 Scheduler grammars and the type-erasure map. The central comparison is between phase-specific admission and one type-uniform directed edge bit.

3 When closure preserves unsafety

3.1 Merged-trace lifting lemma

Let $R \subseteq S \times S$ be a preorder. Consider an original trace and a merged trace that retains every original action in order and inserts closure-required delivery actions. Suppose:

- **initial relation:** $R(s_0, s_0)$;
- **increasing insertion:** for every inserted action b, $R(s, \delta(s, b))$;
- **forward simulation:** for every retained action a, $R(s, t)$ implies $R(\delta(s, a), \delta(t, a))$;
- **transitivity:** R is transitive.

Lemma 1 (merged-trace lifting). If the original and merged traces begin in related states, then their terminal states are related.

Proof. Induct over the merged trace while counting how many original actions have been consumed. At an inserted action, increasing insertion extends the relation and transitivity composes it with the current invariant. At a retained action, forward simulation advances both sides. At the end, all original actions have been consumed, giving the terminal relation. □

3.2 Legal scheduler-closure theorem

Assume additionally that unsafety is upward closed:

$$R(s, t) \wedge \mathsf{Unsafe}(s) \Rightarrow \mathsf{Unsafe}(t).$$

Theorem 1 (legal relational closure preservation). Let $\tau \in L_{G_T}(P, B)$ be unsafe. If the type-erasure transform $C(\tau)$ is a legal target execution in $L_{G_D}(P, B)$ and the four relational premises above hold for the inserted and retained actions of $C(\tau)$, then

$$\mathsf{Unsafe}(\mathrm{last}(C(\tau))).$$

Consequently,

$$O(\mathrm{last}(C(\tau))) \in \mathsf{UnsafeObs}_{G_D}(P, B).$$

Proof. Target legality supplies a valid coarse execution. Lemma 1 gives

$$R(\mathrm{last}(\tau), \mathrm{last}(C(\tau))).$$

Upward closure transfers unsafety to the target terminal state. Applying O yields a member of the target unsafe-observation language. □

The theorem is sufficient, not necessary. It makes clear why "more delivered messages" is not automatically monotone: an inserted message may remove future behavior relevant to the safety witness, so no suitable preorder with upward-closed unsafety exists.

4 Quorum-pivot separation

4.1 Structural criterion

Let P be a bounded certificate-driven locking system with threshold q. Consider two conflicting decision opportunities $C_0 < C_1$ for values 0 and 1. The following premises define the structural mechanism.

1. There are two conflicting certificate opportunities $C_0 < C_1$.
2. A correct identity p is the unique contribution capable of completing both branches inside the bound.
3. Without p, each branch has fixed base support of exactly $q - 1$ contributions.
4. An already-emitted enabling envelope E is processed before C_0 and makes p available to the first branch.
5. An already-emitted repair envelope R is processed after C_0 but before C_1 and makes p unavailable to the second branch.
6. E and R have the same view, sender, and receiver.
7. E and R have distinct phase labels.
8. A phase-specific policy admits E and withholds R.
9. The type-uniform directed grammar has one Boolean bit for that shared edge and therefore admits or rejects E and R together; all other productions, edges, deliveries, and phase order are fixed. Edge off withholds both envelopes, and edge on processes both in the order $E < C_0 < R < C_1$.
10. A correct decision for value i is emitted when the corresponding certificate reaches q, and decisions are irreversible inside the bound.

Theorem 2 (quorum-pivot separation criterion). Every bounded system satisfying Premises 1–10 has an agreement-violating execution under G_T, while no execution in the stated G_D domain violates agreement:

$$\mathsf{UnsafeObs}_{G_T}(P, B) \neq \varnothing, \qquad \mathsf{UnsafeObs}_{G_D}(P, B) = \varnothing.$$

Proof. Under G_T, E is delivered and R is not. The first base of $q - 1$ plus p forms the value-0 certificate and decision at C_0. Since repair is absent, p remains available; the second base plus p forms the conflicting value-1 certificate and decision at C_1.

For G_D, the Boolean domain is exhaustive. If the edge is off, neither E nor R is delivered, so the first branch remains at $q - 1$. If the edge is on, E enables the first certificate, but R is processed before C_1 and disables p; the second branch remains at $q - 1$. Hence neither directed execution contains two conflicting correct decisions. □

This theorem is the intermediate structural result: it identifies a checkable mechanism without tying the statement to one hand-written quorum arithmetic instance.

4.2 Parametric $3f + 1$ family

For every $f \geq 1$, let

$$n = 3f + 1, \qquad q = 2f + 1.$$

Partition identities into B, A, C, and $\{p\}$, with

$$|B| = |A| = |C| = f,$$

where B is Byzantine, A supports only value 0, C supports only value 1, and p is correct. Define

$$Q_0 = B \cup A \cup \{p\}, \qquad Q_1 = B \cup C \cup \{p\}.$$

Then $|Q_0| = |Q_1| = 2f + 1 = q$, while each branch without p has $2f = q - 1$ contributions. The correct intersection of the two quorums is exactly $\{p\}$.

The bounded domain B_f fixes one pair of conflicting values, one shared controlled edge, the order $E < C_0 < R < C_1$, all other messages and edges, once-only delivery, and the absence of late repair after C_1. The only G_D degree of freedom is the Boolean edge bit. Let P_f be the protocol schema with the decisions specified in Premise 10.

Corollary 1 (parametric family). For every $f \geq 1$,

$$\mathsf{UnsafeObs}_{G_D}(P_f, B_f) = \varnothing, \qquad \mathsf{UnsafeObs}_{G_T}(P_f, B_f) \neq \varnothing.$$

The arithmetic is immediate from $2f + 1 = q$ and $2f = q - 1$; the behavioral result follows from Theorem 2.

5 Diagnostic seven-premise boundary

The structural theorem uses semantic premises. For bounded diagnosis and ablation, we encode seven Boolean flags:

- S_1: the first base is one short of quorum;
- S_2: the second base is one short of quorum;
- R_q: the pivot's second-branch participation requires enabling evidence;
- D: repair disables the pivot;
- T: repair is processed before the second opportunity;
- E_d: enabling and repair share the endpoint/view erased by closure;
- X: the fine grammar can distinguish their phase labels.

For delivery bits e and r, the operational unsafe outcome is

$$U(e, r) = (\neg S_1 \vee e) \wedge [\neg S_2 \vee ((\neg R_q \vee e) \wedge \neg(r \wedge D \wedge T))] .$$

Strict separation is

$$\mathsf{Strict} = X \wedge E_d \wedge U(1, 0) \wedge \neg U(0, 0) \wedge \neg U(1, 1).$$

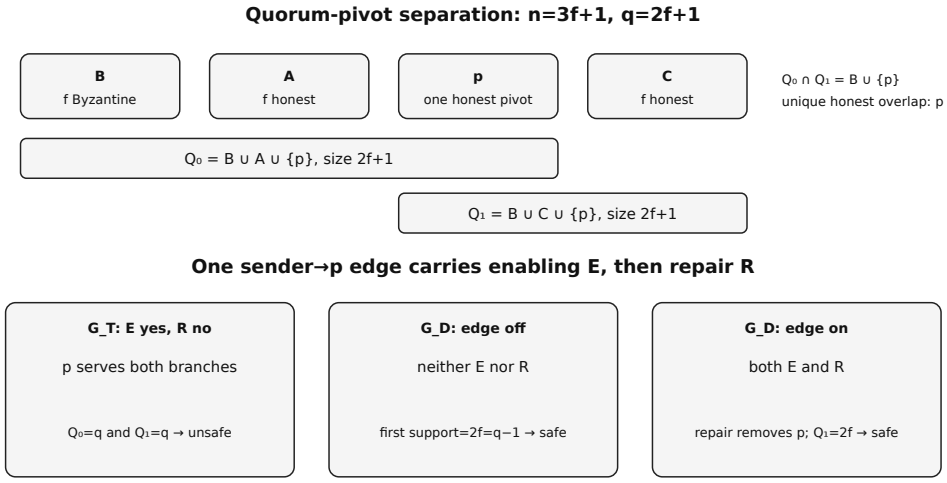


Figure 2 Quorum-pivot mechanism. The fine policy admits enabling evidence but withholds repair; the closed policy either loses the first quorum or repairs the pivot before the second.

Proposition 1 (exact boundary of the diagnostic abstraction). Over all 2^7 assignments,

$$\text{Strict} \iff S_2 \wedge D \wedge T \wedge E_d \wedge X \wedge (S_1 \vee R_q).$$

Derivation. Direct substitution gives

$$U(1,0) = \text{true},$$

$$U(0,0) = \neg S_1 \wedge (\neg S_2 \vee \neg R_q),$$

and

$$U(1,1) = \neg S_2 \vee \neg D \vee \neg T.$$

Therefore

$$\neg U(0,0) = S_1 \vee (S_2 \wedge R_q),$$

$$\neg U(1,1) = S_2 \wedge D \wedge T.$$

Conjoining with $X \wedge E_d$ and simplifying under S_2 yields

$$X \wedge E_d \wedge S_2 \wedge D \wedge T \wedge (S_1 \vee R_q).$$

In the flag order $(S_1, S_2, R_q, D, T, E_d, X)$, the satisfying assignments are exactly

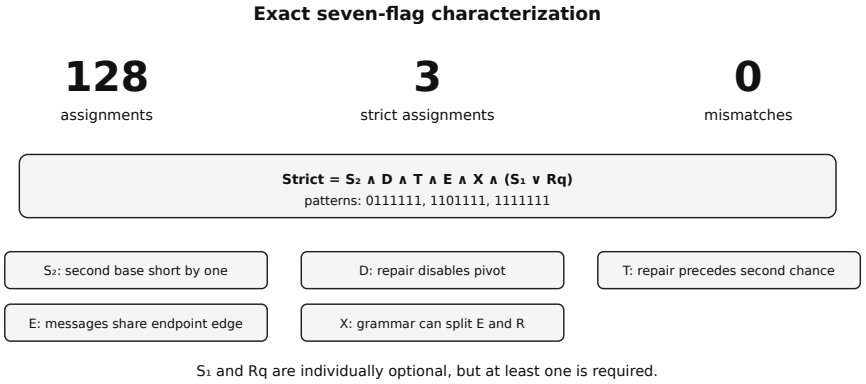


Figure 3 The diagnostic premise boundary. Five premises are always necessary; at least one of first-branch shortage or enabling-dependent reuse must hold.

```
0111111
1101111
1111111
```

The exhaustive truth table is a cross-check of this algebra, not the proof's conceptual core. Proposition 1 characterizes only this seven-flag operational abstraction; it is not a necessary-and-sufficient theorem for all locking protocols.

6 Bounded instantiation to Tendermint-style locking

6.1 Published rule vocabulary

The CometBFT consensus specification organizes each round as `Propose`, `Prevote`, and `Precommit`, followed by `Commit` once a block receives more than two-thirds precommits. A more-than-two-thirds prevote set for a block is a proof-of-lock-change/PoLC; in the normal rule, a validator seeing such evidence locks or relocks the block and precommits it [5]. We use these published rules as a natural transition vocabulary, not as evidence of an implementation bug.

The bounded instance fixes one height H, rounds $r_0 = 0$ and $r_1 = 1$, values 0 and 1, and the same $3f + 1$ validator partition as Section 4. The controlled mutation is `MISSING_LOCK_UPDATE`: after observing a round-0 PoLC for value 0, the pivot emits `PRECOMMIT(0,H,r_0)` but fails to persist `lockedValue=0` and `lockedRound=r_0`.

6.2 Refinement map

The bound fixes every other message, edge, and delivery. It processes the events in the order

$$E < C_0 < R < C_1.$$

There is no late delivery of R after C_1. Thus the type-uniform directed domain has exactly two policies: edge off and edge on. The phase-specific witness admits E and rejects R.

Abstract object	Bounded Tendermint-style rule
pivot p	correct validator required by both conflicting $2f+1$ precommit sets
enabling E	last `PREVOTE(0,H,r0)` from fixed Byzantine sender b_0 to p, completing p's local PoLC
first opportunity C_0	a correct observer receives $B \cup A \cup \{p\}$ precommits and decides value 0
repair R	last `PRECOMMIT(0,H,r0)` from the same b_0 to p, completing p's local commit set
disabled pivot	baseline lock prevents conflicting precommit, or receipt of R makes p commit/advance before C_1
second opportunity C_1	later round-1 value-1 precommit set $B \cup C \cup \{p\}$
shared edge	same H, r_0, b_0, p; labels differ only as `PREVOTE` versus `PRECOMMIT`

Table 1 Refinement map from the structural criterion to the bounded Tendermint-style rule model.

6.3 Result

Proposition 2 (bounded Tendermint-style instantiation). For every $f \geq 1$ in the supplied rule-level model:

- the unmutated locking rule is safe under the phase-specific witness and both directed policies;
- under `MISSING_LOCK_UPDATE`, the phase-specific policy yields conflicting correct decisions;
- under the same mutation, both directed policies remain safe.

The proof is a direct refinement to Theorem 2. With edge off, p never sees the PoLC-completing prevote, so the value-0 certificate is short. With edge on, p sees both the enabling prevote and the commit-completing precommit; the latter is processed before the value-1 opportunity and removes p from the second branch. Under phase-specific admission, the mutation leaves p unlocked and uncommitted after the first branch, permitting the conflicting contribution.

Two independent implementations enumerate the bounded rule system. Python and Go produce identical terminal rows for six combinations of baseline/mutant and scheduler policy for each $f = 1, \ldots, 64$, for 384 cases per implementation. An additional independent checker evaluates the same semantic conditions through $f = 128$. These executions validate the stated refinement; they do not establish conformance of the entire production event loop or the existence of the mutation in CometBFT.

7 Replay erasure as a secondary closure dimension

Phase erasure is the main result. Replay illustrates a different boundary: a repeated byte string may be immediately silent yet alter hidden state that affects future processing.

Let $\pi : S \to S_{\text{rel}}$ project protocol-relevant state, including the set of processed stable message keys. Let $\delta : S \times M \to S$ be the transition function and

$$\mathsf{out} : S \times M \to O^*$$

the protocol-relevant output produced while processing one message.

We require two congruence conditions for every message m:

$$\pi(s) = \pi(t) \Rightarrow \pi(\delta(s, m)) = \pi(\delta(t, m)), \qquad \text{(state congruence)}$$

and

$$\pi(s) = \pi(t) \Rightarrow \mathsf{out}(s, m) = \mathsf{out}(t, m). \qquad \text{(output congruence)}$$

For a marked duplicate whose stable key is already represented in $\pi(s)$, require

$$\pi(\delta(s, m)) = \pi(s) \quad \text{and} \quad \mathsf{out}(s, m) = \epsilon.$$

Theorem 3 (replay erasure with outputs). Under state congruence, output congruence, projected duplicate stutter, and duplicate output silence, erasing all marked duplicates from a finite trace preserves both the final relevant projection and the entire protocol-relevant output trace. Any state-based observation that factors through π is therefore preserved.

Proof. Induct over the original trace and its erased trace while maintaining equality of current projections and accumulated relevant outputs. A marked duplicate leaves the full execution's projection unchanged and appends no output, so the erased side may stutter. For a retained message, state congruence preserves equality of the next projections and output congruence appends equal output fragments. The terminal invariant gives both conclusions. □

The output premise is necessary for the output-trace claim. Equal projections alone permit hidden states to emit different future outputs. State congruence is also necessary: in the counterexample trace `X,X,Y`, the second `X` changes only a hidden bit and produces no output, while `Y` copies that bit into relevant state. The full and deduplicated traces then have different observations. The artifact includes this counterexample and a separate output-congruence negative control.

8 Evidence and formal assurance

8.1 Independent finite checks

The final artifact includes a checker that imports neither the project models nor the proof-certificate kernel. It verifies:

- exact scheduler filtering, no synthesis, once-only delivery, and stable order;
- legal-target relational lifting over all valid two-state finite instances in its scope;
- the structural criterion for quorum thresholds 1 through 64;
- the hidden-state replay counterexample and positive state/output congruence cases;
- the bounded Tendermint-style bridge for $f = 1$ through 128;
- the corrected typing of `UnsafeObs`.

The post-audit SMT package contains positive obligations and damaged controls. In artifact reproduction, Z3 [6] returns `UNSAT` for all eight preservation obligations and `SAT` for all four controls that drop a necessary condition. Go arbitrary-precision arithmetic independently checks the parametric quorum equations.

8.2 Prover developments

The archive contains Lean and Coq sources for scheduler semantics, typed unsafe observations, legal closure lifting, the structural criterion, the parametric family, the Tendermint-style instantiation, replay erasure with outputs, and the diagnostic boundary. Internal proof-obligation identifiers appear only in the artifact map; the paper uses normal theorem numbering.

The primary mature-prover project is pinned to Lean 4.33.0. `lake build`, the per-theorem `#print axioms` audit, source-hygiene checks, and `leanchecker SchedulerClosure` all complete successfully. The Coq development is retained as a secondary archival formalization and is not used to support a stronger assurance claim. The purpose-built certificate checker and SMT obligations provide independent supporting routes, not substitutes for the Lean kernel build.

8.3 Executable validation

Earlier gates supply two finite locking kernels and a fuller two-round event loop with `PROPOSAL`, `PREVOTE`, `POLKA`, `PRECOMMIT`, and `COMMIT`. The baseline is safe across the tested grammars. Controlled obligation violations distinguish phase control, terminal delay, and replay. A Go implementation and an independently written Python mirror agree on canonical terminal cases, while FIFO, LIFO, and seeded event orders preserve the safety classifications. These results test the mechanism and implementation correspondence inside the declared bounds; they are not a statistical estimate of production bug prevalence.

9 Related work and novelty boundary

Twins uses same-identity replicas and round-indexed communication patterns to exercise equivocation, double voting, and state loss [2]. Randomized BFT testing provides message mutation and richer fault exploration [13]. Netrix and ByzzBench provide controlled scheduling and evaluation infrastructure [8, 1]. Mallory uses Lamport-timeline and happens-before feedback to guide distributed-system fault sequences [10]. ByzTwin-Range broadens practical Byzantine experimentation through a digital-twin cyber-range [9]. None of these scheduler, fuzzing, or implementation capabilities is claimed as new here.

Communication closure is the closest conceptual result. It establishes a property-preserving testing reduction for a class of consensus implementations [7]. Our closure operator and boundary differ: phase labels on already-emitted messages are erased to one endpoint bit, and closure-added repair can remove the only correct contribution shared by two conflicting quorum opportunities. Heard-Of and closed message-adversary work already provide directed per-round communication semantics [4, 12], so the graph model itself is not a contribution.

VLSM develops formal validator-system traces and Byzantine/equivocation relationships under validity assumptions [3]. Functional reasoning for distributed systems connects restricted synchronous reasoning to asynchronous operational semantics in Rocq [11]. These works motivate explicit legal-language and refinement obligations. PERF formalizes fault injectors and composition for performance prediction [14], which further rules out a novelty claim based on injector taxonomies.

The table is a capability map, not a quality ranking. The novelty claim is search-bounded: in sources reviewed through 21 August 2026, we found no result combining the same phase-erasure closure, legal-target condition, finite agreement projection, repair/unique-

Work	Part.	Dir.	Phase	Replay	Closure	Sep.	Conf.	Mech.
Twins	Y	part.	–	state	–	–	impl.	–
Comm. closure	–	lossy	–	–	Y	–	impl. cond.	–
HO/msg. adv.	–	Y	–	–	sem.	–	–	–
ByzzFuzz	–	Y	Y	Y	–	–	impl.	–
Netrix	–	Y	Y	Y	–	–	impl.	–
ByzzBench	cfg.	Y	cfg.	cfg.	–	–	impl.	–
Mallory	Y	faults	–	–	–	–	deployed	–
PERF	–	model	faults	model	perf.	–	model/sys.	Maude
This work	case	Y	Y	Y	**Y**	**Y**	bounded rules	Lean

Table 2 Capability map for closest work. 'Closure' is an explicit property-preservation result; 'Sep.' is a safety-observation separation witness; 'Conf.' distinguishes implementation execution from bounded rule-level refinement; 'Mech.' records a mechanized formal model or proof.

pivot criterion, bounded Tendermint-style refinement, exact diagnostic boundary, and replay theorem with both state and output congruence. Absolute novelty is not asserted.

10 Validity and limitations

Internal validity. A shared specification error could affect the formal and executable routes. Mitigations include independent Python and Go models, an independent checker, SMT encodings with damaged controls, exact arithmetic, source-level prover developments, and a safe unmutated baseline. The natural protocol bridge is deliberately small enough to inspect line by line.

Construct validity. Scheduler expressiveness is measured by the set of unsafe terminal observations, not schedule count, code coverage, or bug yield. A different property can induce a different grammar ordering. Type closure is precisely the stable-filter operator of Section 2; systems that permit delay inside a view, phase-specific exceptions, or delivery outside partitions need a separate conformance map.

External validity. The family uses unweighted $3f + 1$ quorums and finite views. The Tendermint-style bridge is rule-level and bounded. Weighted stake, validator-set changes, multi-height state transfer, disk recovery, liveness, and pacemaker behavior are outside the theorem. The bridge does not show that CometBFT contains `MISSING_LOCK_UPDATE`.

Proof assurance. Lean 4.33.0 kernel-checks the primary formalization, and the axiom report lists every dependency reported by `#print axioms`. The independent finite checker, project-local certificate kernel, and SMT controls reduce shared-encoding risk but do not make the bounded claims unbounded. The Coq mirror remains secondary and unbuilt.

Responsible use. All adversarial schedules and mutations run locally. No public network, assets, private keys, denial-of-service traffic, or unauthorized access are involved. A future implementation-level finding should be privately minimized and disclosed before publication of sensitive details.

11 Reproducibility

The artifact exposes one entry point:

```
./reproduce.sh
```

It creates an isolated Python environment, runs the independent proof checks, Python and Go models, cross-language comparison, certificate kernels, SMT obligations and negative controls, installs or activates the pinned Lean toolchain, runs `lake build` and the axiom audit, builds the official LIPIcs source, verifies the page budget and manifests, and fails nonzero if any mandatory stage fails.

The package preserves raw and processed outputs, fixed random seeds, protocol parameters, source registers, tool versions, and SHA-256 manifests. Historical research gates are isolated under `archive/stages/`; active reproduction does not read them.

12 Conclusion

Phase erasure can change more than the number of schedules. When enabling and repair evidence share one endpoint and view, but differ by phase, closing their admission to a single directed edge bit can insert repair before a later certificate opportunity. If repair disables the unique correct quorum pivot, the coarse grammar deletes an agreement-violation witness.

A legal relational theorem identifies when closure is safe; a structural criterion identifies when the locking mechanism defeats that monotonicity; the $3f + 1$ family and bounded Tendermint-style instantiation make the boundary concrete. The diagnostic seven-premise model states the exact local conditions, while the replay theorem shows why future state and output congruence are required for duplicate erasure.

The practical implication is methodological. Testing frameworks should not equate a richer API with a scientific result, nor interpret a smaller scheduler language as property preserving without a closure proof. For a concrete protocol, one must either discharge the legal and relational obligations or exhibit a minimal separation witness and name the missing premise.

13 Acknowledgment and AI-use disclosure draft

The anonymous draft was prepared with substantial assistance from an OpenAI language model for research organization, source discovery, prose drafting, code and artifact generation, and consistency checking. Human authors must independently verify every proof, citation, experiment, and source and remain fully accountable for the manuscript. The model is not an author. This statement must be adapted to the final venue policy.

References

1 Jo ao Miguel Louro Neto and Burcu Kulahcioglu Ozkan. A benchmark framework for byzantine fault tolerance testing algorithms. In *6th International Workshop on Formal Methods for Blockchains (FMBC 2025)*, volume 129 of *Open Access Series in Informatics (OASIcs)*, pages 13:1–13:11, 2025. doi:10.4230/OASIcs.FMBC.2025.13.

2 Shehar Bano, Alberto Sonnino, Andrey Chursin, Dmitri Perelman, Zekun Li, Avery Ching, and Dahlia Malkhi. Twins: BFT systems made robust. In *25th International Conference on Principles of Distributed Systems (OPODIS 2021)*, volume 217 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 7:1–7:29, 2022. doi:10.4230/LIPIcs.OPODIS.2021.7.

3 Miguel Barreto et al. Validating labelled state and message production systems. *arXiv preprint arXiv:2202.12662*, 2022. URL: https://arxiv.org/abs/2202.12662.

4 Bernadette Charron-Bost and André Schiper. The heard-of model: Computing in distributed systems with benign faults. *Distributed Computing*, 22(1):49–71, 2009. doi:10.1007/s00446-009-0084-6.

5 CometBFT Project. Cometbft consensus specification. Release v0.39.3, commit prefix 49b8283, 2026. Engineering specification pinned for the bounded instantiation. URL: https://github.com/cometbft/cometbft/blob/v0.39.3/spec/consensus/consensus.md.

6 Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient smt solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2008)*, volume 4963 of *Lecture Notes in Computer Science*, pages 337–340, 2008. doi:10.1007/978-3-540-78800-3_24.

7 Cezara Drăgoi, Constantin Enea, Burcu Kulahcioglu Ozkan, Rupak Majumdar, and Filip Nikšić. Testing consensus implementations using communication closure. *Proceedings of the ACM on Programming Languages*, 4(OOPSLA):210:1–210:29, 2020. doi:10.1145/3428278.

8 Cezara Drăgoi, Srinidhi Nagendra, and Mandayam Srivas. A domain specific language for testing distributed protocol implementations. In *Networked Systems (NETYS 2024)*, Lecture Notes in Computer Science, pages 100–117, 2024. doi:10.1007/978-3-031-67321-4_6.

9 Tadeu Freitas, Jo ao Soares, and Rolando Martins. Trust, but verify: Byztwin-range, a digital twin cyber-range for byzantine faults. *arXiv preprint arXiv:2604.18049*, 2026. URL: https://arxiv.org/abs/2604.18049.

10 Ruijie Meng, George Pîrlea, Abhik Roychoudhury, and Ilya Sergey. Greybox fuzzing of distributed systems. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 1615–1629, 2023. URL: https://arxiv.org/abs/2305.02601.

11 Haobin Ni, Robbert van Renesse, and Greg Morrisett. Functional reasoning for distributed systems with failures. *arXiv preprint arXiv:2510.12131*, 2025. URL: https://arxiv.org/abs/2510.12131.

12 Kyrill Winkler, Ulrich Schmid, and Yoram Moses. A characterization of consensus solvability for closed message adversaries. In *23rd International Conference on Principles of Distributed Systems (OPODIS 2019)*, volume 153 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 17:1–17:16, 2020. doi:10.4230/LIPIcs.OPODIS.2019.17.

13 Levin N. Winter, Florena Buşe, Daan de Graaf, Klaus von Gleissenthall, and Burcu Kulahcioglu Ozkan. Randomized testing of byzantine fault tolerant algorithms. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1):757–788, 2023. doi:10.1145/3586053.

14 Ziwei Zhou, Si Liu, Zhou Zhou, Peixin Wang, and Min Zhang. A formal framework for predicting distributed system performance under faults. *arXiv preprint arXiv:2602.19088*, 2026. Accepted at FM 2026. URL: https://arxiv.org/abs/2602.19088.